Лабораторно упражнение 7

Създаващи шаблони. Шаблон „Абстрактна Фабрика“

I. Теоретична част

1. Описание на шаблон „Абстрактна Фабрика“

1.1. Цел и мотивация

Цел: Да предостави интерфейс за създаване на семейства свързани или взаимно зависими обекти, без да се уточнява техните типове (конкретните класове).

Мотивация:

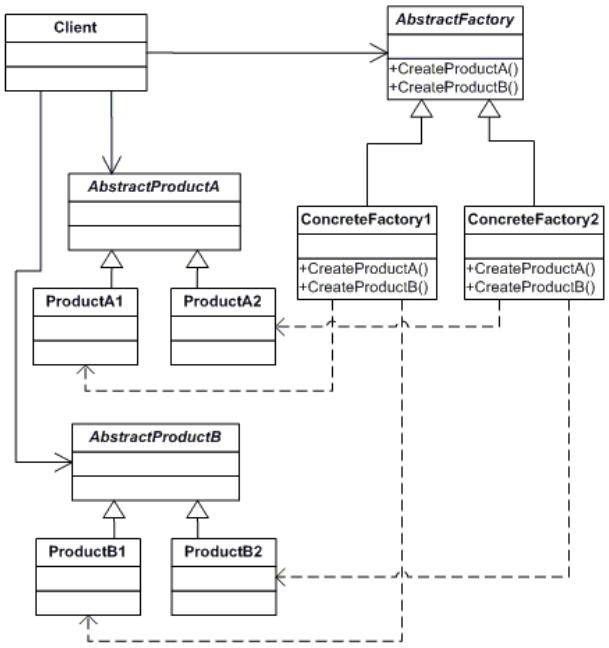
Използването на интерфейси и базови класове позволява абстрахиране от конкретната имплементация, което прави решението просто и разширяемо. На практика това се постига благодарение на полиморфизма. Но някъде в приложението трябва да има отправна точка, в която се създават обектите и съответно, трябва да е известен техния конкретен тип.

1.2. UML диаграма

Диаграмата на класическата „Абстрактна фабрика“ е представена на фигура 4.

В диаграмата участват следните класове и обекти:

- AbstractFactory – интерфейс за операциите, които ще създават абстрактните продукти
- ConcreteFactory – клас, който наследява интерфейса AbstractFactory и имплементира действията, които създават конкретните продукти (обекти от съответните класове)
- AbstractProduct – интерфейс за типа на продукта като за всеки тип продукт се дефинира отделен интерфейс
- Product – клас, който наследява някой от интерфейсите AbstractProduct; обекти от този клас ще бъдат създавани от съответната конкретна фабрика
- Client – приложението, което използва декларираните интерфейси AbstractFactory и AbstractProduct.



Фиг. 4. UML диаграма на шаблон „Абстрактна Фабрика“

2. Пример за реализация на шаблон „Абстрактна фабрика“

Задача: *Да се създаде приложение, което описва особеностите на подвидове животни, които обитават различни континенти. Да се използват животните Слон и Лъв, обитаващи континентите Африка и Азия.*

Примерно решение:

Ще бъдат разгледани най-разпространените видове африкански и индийски слон, както и африкански и азиатски (индийски, персийски) лъв.

Решението минава през следните стъпки:

- Декларират се интерфейсите, които описват Слон и Лъв. Те съдържат методи, които дават информация за съответното животно. Метод `CanMeet` дава връзката между двете животни.

```
interface IElephant
    {
        int GetWeight();
        int GetHeight();
    }
interface ILion
    {
        string GetFeatures();
        void CanMeet(IElephant elephant);
    }
```

- Декларират се класовете, които наследяват двата интерфейса . Това са класовете, описващи африкански и индийски слон, африкански и азиатски лъв.

```
class AfricanElephant:IElephant
    {
        public int GetWeight()
        {
            return 7500;
        }
        public int GetHeight()
        {
            return 4;
        }
    }
    class IndianElephant : IElephant
    {
        public int GetWeight()
        {
            return 6000;
        }
        public int GetHeight()
        {
            return 3;
        }
    }
class AfricanLion : ILion
    {
        public string GetFeatures()
        {
            return "Има голяма грива";
        }
```

```
        public void CanMeet(IElephant elephant)
        {
            Console.WriteLine(this.GetType().Name + " може да срешне " +
elephant.GetType().Name);

        }

    }
   class AsiaticLion:ILion
    {
        public string GetFeatures()
        {
            return "Има малка грива";
        }
        public void CanMeet(IElephant elephant)
        {
            Console.WriteLine(this.GetType().Name + " може да срешне " +
elephant.GetType().Name);

        }
    }
```

- Декларира се интерфейсът, който съответства на абстрактната фабрика и ще създава обекти от продуктовите серии. В случая Лъв и Слон.

```
interface IContinentalFactory
    {
        ILion CreateLion();
        IElephant CreateElephant();
    }
```

- Декларират се класовете, съответстващи на двете различни фабрики (`AfricanFactory, AsiaticFactory`). Това са класовете, който създават обектите

```
class AfricanFactory:IContinentalFactory
    {
        public ILion CreateLion()
        {
            return new AfricanLion();
        }
        public IElephant CreateElephant()
        {
            return new AfricanElephant();
        }
    }
    class AsiaticFactory : IContinentalFactory
    {
        public ILion CreateLion()
        {
            return new AsiaticLion();
        }
        public IElephant CreateElephant()
        {
            return new IndianElephant();
        }
    }
```

- Създава се клиентския клас

```
class Client
    {
        private ILion lion;
        private IElephant elephant;
        public Client(IContinentalFactory factory)
        {
```

```
            lion = factory.CreateLion();
            elephant = factory.CreateElephant();
        }

        public void Run()
        {
            Console.WriteLine(elephant.GetType().Name + " тежи " +
elephant.GetWeight()+"кг");
            Console.WriteLine(elephant.GetType().Name + " е висок " +
elephant.GetHeight()+"м");
            Console.WriteLine(lion.GetType().Name + lion.GetFeatures());
            lion.CanMeet(elephant);
        }
    }
```

- Създава се кодът на главния метод. В него се посочва с коя от фабриките ще се работи.

```
class Program
    {
        static void Main(string[] args)
        {
            IContinentalFactory factory1 = new AfricanFactory();
            Client client1 = new Client(factory1);
            client1.Run();

            IContinentalFactory factory2 = new AsiaticFactory();
            Client client2 = new Client(factory2);
            client2.Run();
        }
    }
```

II. Задачи за изпълнение

1. Да се разгледа примерната реализация, представена в т. 2. Да се направи модификация на програмния код като се добавят нови методи към интерфейсите ILion и IElephant, които дават допълнителна информация като местообитание, латинско наименование, начин на хранене и др.
2. Да се направи вариант на задачата в т. 2 като за реализацията се използват абстрактни класове.
3. Като се използва шаблон „Абстрактна Фабрика“ да се създаде приложение, което работи със класове, описващи следните геометрични фигури: окръжност, правоъгълник и квадрат. Фигурите могат да са от два цвята – син и червен.